

JavaScript Interview Cheat Sheet (Visual & Condensed)

Event Loop

Call stack + micro/macro task queues. Microtasks run first.

Closures

Functions remember outer scope. Used for privacy & factories.

Prototype Chain

Objects inherit via `__proto__` lookup chain.

`==` vs `===`

`===` strict; `==` coerces types.

Hoisting

`var` hoisted as undefined; `let/const` in TDZ.

Pass by Value/Reference

Primitives by value; objects reference copy.

Debounce/Throttle

Debounce waits; throttle limits frequency.

this

Depends on call-site; arrow functions lexical.

Map/Set/WeakMap/WeakSet

Weak collections allow GC of keys.

Currying

Convert `f(a,b)` into `f(a)(b)`.

Memoization

Cache expensive results.

Deep vs Shallow Copy

Shallow copies first level; deep copies all.

Event Delegation

Use parent listener + bubbling.

Polyfills

Custom implementations of map/filter/reduce.

Promises

pending→settled; immutable; chainable.

Promise.all/race

all waits; race returns first result.

Async/Await

Syntactic sugar over promises.

Generators

yield to pause/resume.

V8 Engine

AST→bytecode→optimized machine code.

Single-threaded JS

Avoids DOM race conditions.

TDZ

let/const inaccessible before init.

Optional Chaining/Nullish

`obj?.a; x ?? y.`

Modules

CommonJS require vs ESM import.

Immutable vs Mutable

Objects mutable; primitives not.

Tail Call Optimization

Reuse stack for tail calls.